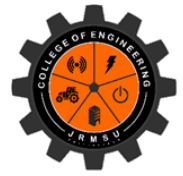




Republic of the Philippines

JOSE RIZAL MEMORIAL STATE UNIVERSITY

Gov. Guading Adaza St., Sta. Cruz, Dapitan City, Philippines



COLLEGE OF ENGINEERING

GSU Gateway Technical Documentation

General Services Office Maintenance Request System

Submitted by:

CHERRYLOU URBANO

APRIL GRACE LARANJO

SHIENNA BELLE ELOPRE

CHRISTIAN VIDAD

NICE MIA LAGAS

Students

I. Overview

GSU Gateway is a mobile application designed for Android, iOS, and Web platforms, developed specifically for the General Services Office (GSO) of Jose Rizal Memorial State University (JRMSU). The system serves as a centralized digital solution that manages the complete lifecycle of facility and equipment maintenance requests within the university. By bringing all stakeholders onto a single platform, GSU Gateway modernizes how the GSO operates and communicates with the rest of the university community.

Prior to the development of this system, all maintenance requests were handled manually through paper forms or walk-in submissions. This traditional approach proved to be highly inefficient, as it offered no real-time visibility into the status of any given request. Requesters had no way of knowing whether their submissions had been received, reviewed, or acted upon, leaving them without any reliable means of follow-up or tracking.

The absence of a structured digital process also created significant accountability gaps within the approval workflow. There was no clear record of who approved or disapproved a request, nor was there a timestamp indicating when decisions were made. This lack of an audit trail made it difficult to identify bottlenecks, resolve disputes, or ensure that proper procedures were being followed at every level of the organization. Coordination between GSO staff and requesters was equally problematic, particularly when it came to scheduling maintenance work. Without a centralized system, arranging and confirming schedules required additional back-and-forth communication, which further delayed the resolution of maintenance concerns. These compounding inefficiencies ultimately affected the overall quality of service that the GSO was able to provide.

GSU Gateway was built to resolve all of these challenges in one unified solution. It enforces a strict, sequential multi-level approval chain that ensures every request passes through the proper channels before action is taken. Real-time status tracking keeps requesters informed at every stage, while push notifications alert stakeholders whenever a key event occurs such as an approval, rejection, or schedule assignment. Administrators also have full control over user account registration and management, ensuring that access to the system remains secure, organized, and properly governed.

II. Requirements and Features

2.1 User Roles/Levels

1. *Requester*

- Allows requester to submit maintenance service requests to the system. This action initiates the maintenance workflow.
- Requester Submits feedback

2. *Staff*

- Staff members can track the status of service requests submitted by requester. This provides visibility on request progress and updates.
- Staff verifies/denies the maintenance request.
- Staff will assigned maintenance schedule if the campus director approved the request.

3. *Head*

- The department head is responsible for reviewing and approving or rejecting submitted service requests first. This step is critical for ensuring appropriate prioritization and authorization.

4. *Campus Director*

- The Campus Director is responsible for reviewing and approving or rejecting submitted service requests after head. This step is critical for ensuring appropriate prioritization and authorization.

4. *Admin*

- Admin validate and approve new user accounts before granting access to the system.
- Admin can remove existing user accounts from the system when they are no longer needed or violate policies.
- Admin can update user information such as name, role, or department affiliation, maintaining accurate and up-to-date user data.

2.2 System Requirements

2.2.1 Functional Requirements

1. User Registration:

- New university personnel register by providing their personal details, office, position, role, and account credentials. Submitted accounts are placed in a pending state and cannot be used until reviewed and approved by a System Admin.

2. User Authentication:

- Users log in using their registered username and password, and the system issues a Bearer token upon successful authentication to maintain the session. The session persists across app restarts until the user logs out, and a clear error message is shown if credentials are invalid or the account is inactive.
- Users can request a reset link via email if they forgot their password.
- Users can sign in using their Google accounts(email).

3. Role-Based Access Control:

- Each user role System Admin, Head, Staff, Requester, and Campus Director has access only to the screens and actions relevant to their responsibilities. The system dynamically adjusts navigation menus, dashboards, and available buttons based on the logged-in user's role.

4. Maintenance Request Submission:

- Requesters submit a maintenance concern by selecting a maintenance type, entering the location, and providing a description of the issue all fields are required. Upon successful submission, the requester is shown a confirmation screen outlining the next steps in the approval workflow.
- Requesters can attach photos to a maintenance request.

5. Sequential Multi-Level Approval:

- Every maintenance request must pass through three approval levels in strict order: Staff verification, Head approval, then Campus Director final approval. No level can be skipped, and each step must be completed before the request advances to the next.

6. Priority Assignment:

- After a request receives full approval from all three levels, a Staff member assigns a priority number to determine the order in which it will be addressed. The system can auto-generate a suggested priority number based on the maintenance type, or the Staff member may enter one manually.

7. Schedule Assignment:

- Staff assigns a maintenance date and time slot to a fully approved request using a calendar interface that disables past dates. Available time slots run from 7:00 AM to 4:00 PM in hourly increments, excluding 12:00 PM, and the requester is notified via push notification once a schedule is set.

8. Mark as Done:

- Staff marks a scheduled request as Done after the maintenance work has been completed, which updates the request status and notifies the requester. This action also unlocks the feedback feature, allowing the requester to rate and review the service.

9. Disapproval and Denial:

- Staff may deny a request before verification, while the Head or Campus Director may disapprove it at their respective approval stages all of which require a written reason before the action can be submitted. Once rejected, the request enters a terminal state and cannot be reopened, and the requester is notified with the reason provided.

10. Dashboard KPI Overview:

- Each role sees a tailored dashboard displaying relevant statistics the System Admin sees account KPIs, while all other roles see request KPIs such as Total, Pending, Approved, Done, and Disapproved counts. A recent activity feed and quick action shortcuts are also displayed below the KPI cards.

11. User Account Management:

- The System Admin can view, edit, and delete any registered user account from the User Management screen. Editable fields include name, email, contact number, and username, and all changes are saved to the server with success or error feedback.

12. Account Approval:

- All newly registered accounts require System Admin approval before the user can log in. The admin may approve an account to activate it immediately, or reject it with a required written reason, after which the account is marked as Disapproved.

13. Push Notifications:

- The system sends native push notifications to users at key events in the workflow, such as when a request is verified, approved, scheduled, or marked as done. Tapping a notification opens the relevant screen within the app for immediate context.

14. Feedback Submission:

- Requesters can submit feedback on completed requests by selecting a star rating from 1 to 5 and writing a required comment. Only requests with a Done status and

no existing feedback are eligible, and a confirmation screen is shown upon successful submission.

15. Notification Inbox:

- All users have access to a notification inbox that displays their full notification history in reverse chronological order, with unread items visually distinguished from read ones. Users can tap any notification to view its details, or use the "Mark All as Read" button to clear all unread items at once.

16. Profile Management:

- All users can view and update their profile details, including name, email, contact number, and department. Username and role are read-only fields, and any saved changes are synced to both the server and local device storage.

17. In-App User Manual:

- All users can access a step-by-step guide with screenshots inside the app accessible from the dashboard menu.

18. AI Assistant Bot:

- A chat assistant helps requesters improve their description when filling out the form.SMS Notification

19. SMS Notification

- Staff can send an SMS message directly to a user from within the app

20. Login Location Tracking:

- The app records the user's device location upon login

2.2.2 Non-Functional Requirements

1. Cross-Platform: The application is built from a single codebase that runs on Android, iOS, and the web, ensuring consistent functionality and user experience across all platforms.

2. Offline Resilience: When the server is unreachable or there is no internet connection, the app displays a clear error message instead of crashing, keeping the application stable under poor network conditions.

3. Security: All API requests are authenticated using Bearer tokens stored in the device's AsyncStorage, and the token is cleared immediately upon logout to prevent unauthorized access.

4. Performance: Every API call is configured with a 45-second timeout enforced by an AbortController, preventing the app from hanging on slow or unresponsive network connections.

5. Usability: Navigation and action buttons are rendered based on the user's role, ensuring users are never presented with options that do not apply to them, keeping the interface clean and intuitive.

6. Auditability: All approval-related actions verifications, approvals, disapprovals, and denials are recorded with the responsible user and timestamp, creating a reliable audit trail for every maintenance request.

7. Bearer tokens in AsyncStorage: Add Google OAuth tokens are also handled via expo-auth-session.

8. Privacy / Data Collection: The app records the user's login location via expo-location. Users should be informed and consent must be obtained per platform policy.

9. AI Integration: The AI Assistant sends form data to a backend /ai-assist endpoint. Response quality depends on the backend AI service's availability.

10. File Upload: Maintenance requests support image attachments via expo-image-picker; file size and format handling applies.

III. Technology Stack

Frontend Mobile App

Component	Technology
Framework	React Native 0.81.5
Platform SDK	Expo~54
Routing	Expo Router v6(file-based)
Push Notifications	Expo-notifications + Firebase(FCM)
Language	JavaScript(JSX) + TypeScript
Google Authentication	expo-auth-session ~7.0.11 + expo-web-browser ~15.0.11 + expo-crypto ~15.0.9
Location Services	expo-location ~19.0.8
Image Picker	expo-image-picker ~17.0.10

Backend

Framework	Laravel(PHP)
Server	php artisan server -- host=0.0.0.0 -- port=8000
Authentication	Laravel Sanctum(Bearer token)
Database	MySQL(via XAMPP/Laragon)

Dev and Networking Tools

Tool	Purpose
Tailscale	VPN mesh shares backend between dv machines
ngrok	Public tunnel for remote testers
Expo Dev Client	Custom native build(required for Firebase push)
EAS Build	Builds distributable .apk for android

IV. System Architecture

Context

To build an efficient, scalable, and maintainable mobile application for the General Services Office of Jose Rizal Memorial State University, ensure comprehensive request tracking, facilitate real-time notifications, streamline the approval workflow, and optimize overall maintenance management, the team has chosen specific technologies based on performance, ease of use, community support, and industry standards.

Technologies

The team will use the following technologies for front-end mobile development, quality assurance (QA) testing, backend development, documentation and deployment, and project management:

Front-End Mobile Development

- **React Native:** A cross-platform mobile framework that allows building Android and iOS applications from a single codebase using JavaScript and React.
- **Expo:** A development platform built on top of React Native that simplifies building, testing, and deploying mobile applications without requiring native code configuration.
- **Expo Router:** A file-based routing library for React Native that organizes navigation by folder and file structure, making the codebase easier to maintain.
- **TypeScript:** A statically typed superset of JavaScript that catches errors during development and improves code quality and readability.
- **React Native Reanimated:** An animation library for React Native that provides smooth, performant UI transitions and gesture-based interactions.
- **Figma:** A collaborative wireframing and UI design tool used for designing the application's screens and prototypes before implementation.

QA Testing

- **Expo Go:** A mobile client used during development to instantly preview and test the application on a physical device without requiring a full build.
- **Postman:** For API testing to verify that backend endpoints return correct responses and behave as expected.
- **Manual Device Testing:** Physical Android devices are used for testing real-world behavior including push notifications, gestures, and performance under actual network conditions.
- **EAS Build (Expo Application Services):** Used to generate production-ready APK builds for internal testing and final deployment validation.

Backend Development

- **PHP (Laravel):** A widely used server-side scripting language and framework known for its simplicity, flexibility, and extensive community support. Laravel provides built-in tools for routing, authentication, and database management, making it ideal for building RESTful APIs that power the mobile application.
- **MySQL:** A reliable and widely adopted relational database management system used to store all application data including users, maintenance requests, schedules, and notifications.
- **Laravel Sanctum:** A lightweight authentication package for Laravel that issues Bearer tokens used to authenticate API requests from the mobile application securely.
- **XAMPP:** A lightweight, open-source software package that provides a local development environment including Apache, MySQL, and PHP. It allows the team to run and test the backend server locally before deployment.
- **Firebase Cloud Messaging (FCM):** A cross-platform messaging service used to deliver push notifications to Android devices. It works together with the Expo Push Notification Service to notify users of real-time updates on their maintenance requests.

Documentation and Deployment

- **Expo Application Services (EAS):** A cloud build and deployment service for Expo projects. It handles generating APK and IPA build files for Android and iOS distribution without requiring a local build environment.
- **Git / GitHub:** A version control platform used to manage the project's source code, track changes, and collaborate across the development team.
- **GitHub (Documentation Hosting):** Project documentation is hosted and shared via GitHub making it accessible to all team members and stakeholders.
- **Microsoft Word / Google Docs:** Used for writing and drafting project documentation including this technical document, user manuals, and system design files.
- **REST API endpoints of the GSU Gateway system:** The API collection is published via a public URL accessible to the development team and is also exported as a JSON file stored in the project's GitHub repository.

Project Management

- **Trello:** Used as the primary project management tool for tracking all development tasks, user stories, backlogs, work in progress, and completed items using a Kanban-style board.
- **Notion:** Used as a centralized workspace for organizing and maintaining project documentation.
- **Facebook Messenger (Group Chat):** Used for day-to-day team communication, quick updates, task coordination, and sharing progress among team members during the development process.

Rationale for Choosing this Technologies

React Native 0.81.5 - Chosen for its ability to build a single codebase that runs on both Android and iOS, reducing development time without sacrificing performance.

Expo (~54) - Selected to simplify project setup and provide ready-to-use tools such as push notifications and build services, allowing the team to focus on building features rather than managing configurations.

Expo Router v6 (File-Based Routing) - used because its file-based structure organizes screens intuitively, making navigation easier to manage as the number of screens and user roles grew.

Expo Notifications and Firebase (FCM) - Chosen to deliver real-time push notifications to users. Expo Notifications handles the app-side logic while Firebase FCM handles actual delivery to Android devices.

JavaScript (JSX) & TypeScript JSX - was used for building UI components due to its compatibility with React Native, while TypeScript was added for static type checking to catch errors early and improve code maintainability.

Laravel (PHP) - Selected for its clean structure, built-in authentication tools, and seamless

MySQL integration, making it practical for building the backend REST API.

php artisan serve (host=0.0.0.0, port=8000) - used to run the Laravel development server accessible to all network interfaces. The backend is hosted on a team member's local machine and accessed by the mobile app through its Tailscale IP address, allowing the team to share a single backend instance during development without deploying to a live server

Laravel Sanctum (Bearer Token) - Chosen for its lightweight token-based authentication, ensuring only authenticated users can access protected API endpoints.

MySQL (via XAMPP / Laragon) - Used for its reliability and compatibility with Laravel. XAMPP and Laragon provided a simple local server environment during development.

Tailscale - used as a VPN mesh tool so team members on different networks could share access to the same local backend server without deploying to a live host.

ngrok - used to expose the local backend to the internet for testing API connections and push notifications from devices outside the local network.

Expo Dev Client - Required to create a custom native build that supports Firebase push notifications, which are not available in the standard Expo Go app.

EAS Build - used to generate a distributable APK that team members and testers could install directly on their Android devices.

V. API Endpoints

Base URL: http://<SERVER_IP>:8000/api

All protected endpoints require:

Authorization: Bearer <token>

Accept: application/json

Content-Type: application/json

Authentication

POST /login - Authenticate user and receive token (Public)

POST /register - Register new user account (Public)

POST /logout - Invalidate token and end session (Authenticated)

POST /forgot-password - Request password reset link (Public)

POST /reset-password - Submit new password with token (Public)

POST /auth/google/register - Register or login via Google OAuth (Public)

GET /auth/check-username - Check username availability during signup (Public)

Maintenance Requests

GET /maintenance-requests - Get list of maintenance requests (Authenticated)
POST /maintenance-requests - Submit a new maintenance request (Requester)
PATCH /maintenance-requests/{id} - Update request status (Head / Director)
GET /maintenance-requests/list-with-details - Get requests with full detail (Authenticated)
POST /maintenance-requests/{id}/assign-schedule - Assign schedule + priority (Staff)
POST /maintenance-requests/{id}/cancel - Cancel a request (Requester)
POST /staffpov/{id} - Staff verify/deny action (Staff)
PATCH /headpov/{id} - Head approve/disapprove action (Head)
PATCH /directorpov/{id} - Director approve/disapprove action (Director)

Maintenance Types

GET /maintenance-types - Get list of available maintenance types (Authenticated)

Schedule

POST /maintenance-requests/{id}/assign-schedule - *Assign a schedule to an approved request (Staff)*
GET /generate-priority-number/{maintenanceTypeId} - Get suggested priority (Staff)

Feedback

POST /feedback - Submit feedback for a done request (Requester)
GET /feedbacks - Get all feedbacks (Authenticated)
GET /feedbacks/{id}/details - Get feedback details (Authenticated)

Profile & Settings

GET/PATCH /profile - View/update own profile (Authenticated)
GET/POST /settings - View/update app settings (Authenticated)
POST /settings/toggle-location-tracking - Toggle location tracking on/off (Authenticated)

Location

POST /login-locations - Record login location (Authenticated)

Users

GET /users-list - Get list of all user accounts (System Admin)
PATCH /users/{id} - Edit user details (System Admin)
GET /users/reqInfo - Get requester info (Authenticated)
PATCH /users/{id}/updateAccountStatus (approve)
PATCH /users/{id}/disapproveAccountStatus (reject) - *Approve or reject a user account (System Admin)*
POST /users/push-token - Register device push notification token (Authenticated)

Notifications

GET /notifications - Get all notifications for the user (Authenticated)
POST /notifications/{id}/read - Mark a specific notification as read (Authenticated)

POST /notifications/read-all - Mark all notifications as read (Authenticated)

Dashboard

GET /dashboard-stats - Get KPI statistics for the dashboard (Authenticated)